



UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

SEDE REGIONAL DE CHIQUIMULA

TÍTULO:

DOCUMENTACION. PROYECTO PROGRAMACION

ALUMNO:

CRISTIAN FOLGAR	1390-25-6993
BRANDON RAMIREZ	1390-25-6988
ELDER MENDEZ	1390-25-8932
CARLOS RAMOS	1390-25-11203

CATEDRÁTICO:

KELLER OBDULIO MATTA CALDERON

CÁTEDRA:

PROGRAMACIÓN I TERCER SEMESTRE

SECCIÓN “A”

DOCUMENTACIÓN DE ARQUITECTURA Y DISEÑO DE SOFTWARE

Sistema de Gestión de Inventario y Ventas

1. DESCRIPCIÓN GENERAL DEL SISTEMA

El **Sistema de Inventario y Ventas** es una solución de software orientada a entornos de escritorio, diseñada con el objetivo de centralizar, automatizar y optimizar las operaciones logísticas y comerciales de una empresa. El sistema permite el control estricto de existencias (stock), la administración de carteras de clientes, la automatización del proceso de facturación y el desglose de reportes financieros al cierre de la jornada.

A nivel de ingeniería, el software destaca por abandonar los paradigmas monolíticos y adoptar una arquitectura modular orientada a objetos. Esto permite que el sistema procese flujos de datos masivos con un consumo de recursos de hardware mínimo y con una actualización visual en tiempo real.

2. STACK TECNOLÓGICO Y JUSTIFICACIÓN TÉCNICA

Para garantizar un rendimiento óptimo, estabilidad y una interfaz de usuario fluida, el equipo seleccionó las siguientes tecnologías:

- **Lenguaje de Programación C++ (Estándar C++11/C++14):** Elegido por su velocidad de ejecución nativa, control directo sobre la memoria y alta portabilidad en sistemas de escritorio.
- **Dear ImGui (Immediate Mode GUI):** Framework para el desarrollo de la interfaz gráfica de usuario. Al operar bajo el paradigma de "modo inmediato", los componentes visuales se destruyen y renderizan dinámicamente frame por frame, eliminando la sobrecarga de memoria que sufren los frameworks basados en modo retenido tradicionales.
- **GLFW3 y OpenGL 3:** APIs de bajo nivel encargadas de la abstracción del sistema operativo. GLFW gestiona la creación de la ventana nativa y los eventos de entrada (teclado y mouse), mientras que OpenGL 3 procesa los gráficos mediante aceleración por hardware directamente en la GPU.
- **Microsoft Visual C++ (MSVC / Kit de Herramientas 2022):** Entorno de compilación nativo que optimiza el enlazado estático de dependencias a través del archivo de proyecto SistemaInventarioVentasGUI.vcxproj.

3. ARQUITECTURA DEL SOFTWARE Y PATRONES DE DISEÑO

3.1 Patrón de Diseño DAO (Data Access Object)

El núcleo del sistema implementa de forma estricta el patrón **DAO**. Su propósito fundamental es crear un aislamiento total entre la **Capa de Presentación (Interfaz Gráfica)** y la **Capa de Persistencia (Almacenamiento de Datos)**.

La lógica visual (AppUI) no conoce cómo se guardan o modifican los datos internamente; únicamente invoca funciones genéricas del DAO. Esto permite que, si el sistema migra de una base de datos local a una base de datos en la nube en el futuro, solo se requiera modificar el archivo DAO sin alterar una sola línea de código de las ventanas gráficas.

3.2 Principios de Diseño de Software Aplicados (SOLID)

Durante el desarrollo del código modular, el equipo aplicó principios fundamentales de ingeniería de software para evitar el acoplamiento:

1. **Principio de Responsabilidad Única (Single Responsibility Principle - SRP):** Cada clase tiene una sola razón para cambiar. Por ejemplo, la clase Producto solo almacena datos; la clase ProductoDAO solo interactúa con la base de datos, y la clase AppUI solo se encarga de pintar la pantalla.
2. **Separación de Intereses (Separation of Concerns):** Se crearon carpetas independientes (include/ y src/) y capas bien delimitadas (Entidades, DAOs, UI y Estructuras) para que los cambios en un módulo no generen efectos secundarios o errores en cascada en los demás.

4. DESGLOSE MODULAR Y DICCIONARIO DE CLASES

A continuación, se detalla el plano arquitectónico del sistema. El código fuente fue dividido de manera orientada a objetos en las siguientes cuatro capas:

4.1 Capa de Entidades (Modelos de Datos)

Son las clases puras que representan los objetos reales de la lógica del negocio. Actúan como contenedores de variables.

- Producto.h / .cpp: Atributos del artículo (id, nombre, precio, stock, categoria).
- Cliente.h / .cpp: Atributos para la identificación comercial (nombre, identificacion/NIT, telefono).
- Usuario.h / .cpp: Atributos para credenciales de acceso y niveles de operador (usuario, password, rol).
- Venta.h / .cpp: Atributos globales de la transacción de salida (idFactura, fecha, total).

4.2 Capa DAO (Acceso y Sincronización de Datos)

Clases encargadas de encapsular las consultas o manipulaciones de los datos.

- ConexionBD.h / .cpp: Maneja los estados de conexión física hacia el motor de almacenamiento.
- ProductoDAO.h / .cpp: Contiene los algoritmos de persistencia para el inventario.
- ClienteDAO.h / .cpp: Administrador de registros para la cartera de clientes.
- VentaDAO.h / .cpp: Controla la inserción de la factura y la reducción automática de existencias.
- CorteDAO.h / .cpp: Ejecuta filtros y sumatorias para los reportes de cierre de caja.

4.3 Capa de Control y Estructuras de Datos

Módulos destinados a optimizar el rendimiento y flujo en memoria.

- ListaProductos.h / .cpp: Clase contenedora que gestiona la búsqueda, indexación y ordenamiento de artículos en tiempo de ejecución.
- ColaClientes.h / .cpp: Implementación exacta de una estructura de datos lineal **FIFO (First In, First Out)**. Su objetivo es organizar de forma secuencial las colas de espera físicas de los clientes en los puntos de venta.

4.4 Capa de Interfaz Gráfica (UI)

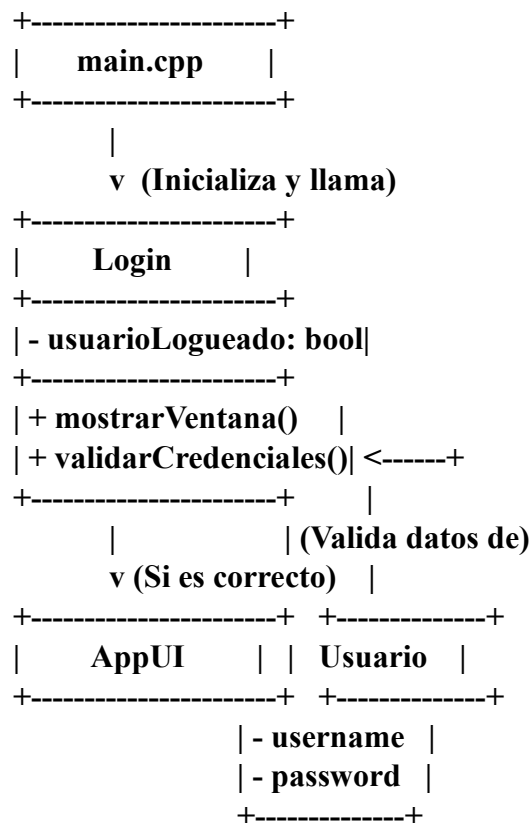
Clases encargadas del ciclo de renderizado por segundo.

- **main.cpp:** Inicializa los contextos de GLFW/OpenGL, carga las configuraciones core de ImGui y mantiene el ciclo infinito de actualización (*Game Loop*).
- **Login.h / .cpp:** Controla de forma aislada la seguridad del formulario de ingreso, ocultando caracteres y validando credenciales.
- **AppUI.h / .cpp:** Diseña el menú global de navegación, las botoneras interactivas y renderiza tablas de datos legibles a través de buffers dinámicos.

5. DIAGRAMAS Y FUNCIONES PRINCIPALES POR MÓDULO

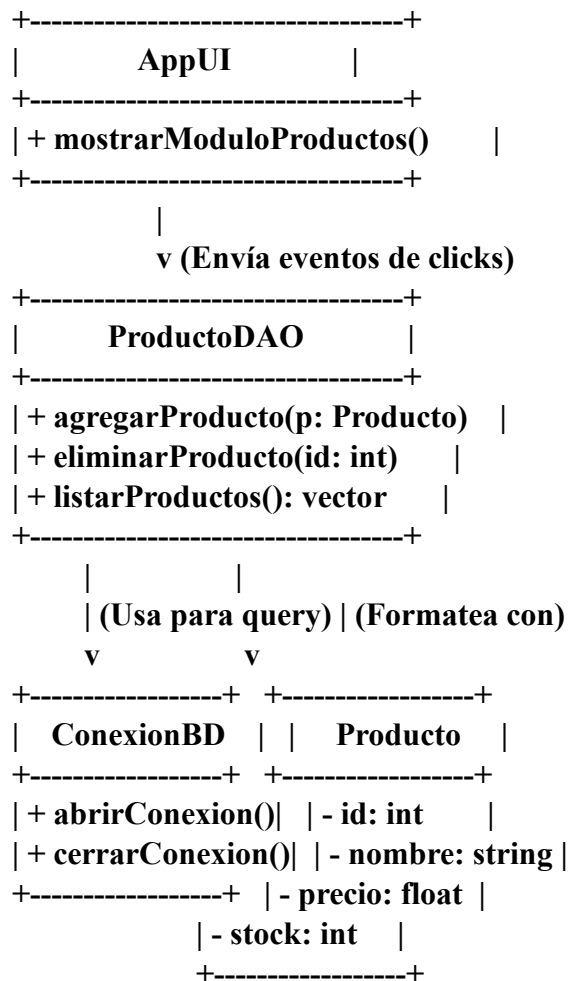
5.1 Módulo de Autenticación y Seguridad (Login)

- **Descripción:** Bloquea el acceso al sistema hasta validar que el operador cuente con credenciales registradas. Cleans buffers al cerrar sesión.
- **Funciones Principales:**
 - **Login::mostrarVentana():** Dibuja la ventana y los inputs protegidos en pantalla.
 - **Login::validarCredenciales(usuario, password):** Compara los datos ingresados con los registros de la entidad Usuario.
 - **Login::isLogueado():** Retorna una bandera booleana (true/false) para liberar o bloquear el flujo del menú principal.



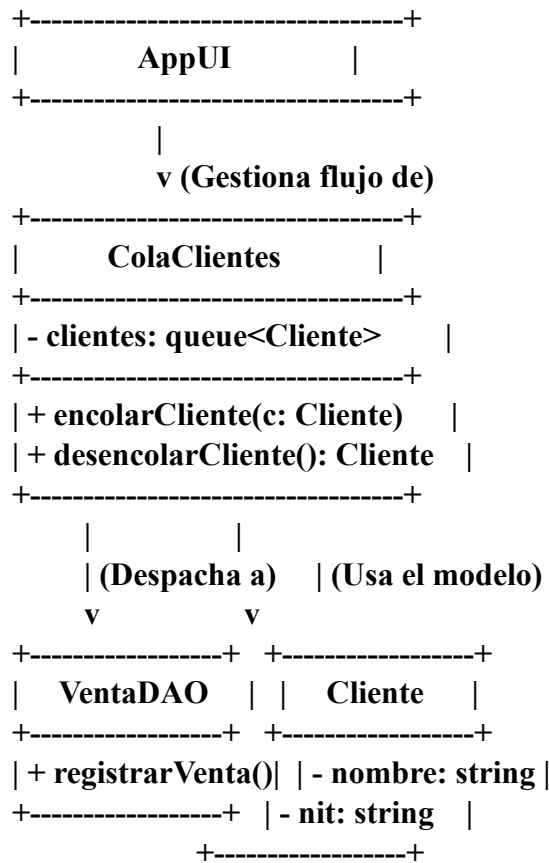
5.2 Módulo de Gestión de Inventario (Productos)

- **Descripción:** Permite administrar las existencias del almacén a través de una interfaz interactiva CRUD.
- **Funciones Principales:**
 - `AppUI::mostrarModuloProductos()`: Renderiza el formulario de captura y la tabla de datos.
 - `ProductoDAO::agregarProducto(Producto nuevo)`: Recibe la entidad validada y la inserta en el almacenamiento.
 - `ProductoDAO::eliminarProducto(int id)`: Busca el artículo mediante su identificador único y lo remueve del sistema.
 - `ListaProductos::actualizarTabla()`: Sincroniza el vector intermedio en memoria con la tabla visual de ImGui de forma instantánea.



5.3 Módulo de Atención y Ventas (En Desarrollo)

- **Descripción:** Diseñado para agilizar el despacho de facturas y controlar el flujo de clientes concurrentes en el establecimiento.
- **Funciones Principales:**
 - ColaClientes::encolarCliente(Cliente c): Inserta un nuevo cliente al final de la cola de espera de atención.
 - ColaClientes::desencolarCliente(): Procesa y remueve al cliente que está al inicio de la fila (Estructura FIFO).
 - VentaDAO::registrarTransaccion(Venta v): Consolida la factura e impacta el stock del módulo de inventario.



6. CONCLUSIONES

- **La pantalla es rápida y no traba la computadora:** Al usar Dear ImGui y OpenGL 3, la interfaz gráfica se dibuja directo en la tarjeta de video cuadro por cuadro. Esto hace que el programa consuma una cantidad mínima de memoria RAM y que las tablas de productos se actualicen al instante sin ralentizar el sistema.
- **El código quedó limpio gracias al Patrón DAO:** Separar los archivos visuales de los archivos que manejan los datos fue la mejor decisión del equipo. Como la pantalla (AppUI) no toca directamente la información, el programa es súper ordenado. Si en el futuro decidimos conectar una base de datos real con MySQL o SQLite, solo tendremos que programar el archivo ConexionBD sin modificar ni una sola línea del diseño de las ventanas.
- **Las colas de clientes resuelven un problema real:** El uso de la estructura ColaClientes con el principio FIFO (Primero en entrar, primero en salir) nos dio la lógica exacta para simular una fila de espera en la vida real. Esto asegura que el área de cajas atienda a los clientes en el orden correcto en el que llegaron, evitando desorden en el flujo de ventas.
- **Trabajo en equipo eficiente:** Dividir el proyecto en carpetas específicas (include/ para declaraciones y src/ para la lógica) nos permitió trabajar en grupo sin pisarnos el código. Mientras unos maquetaban el diseño de los botones en la interfaz, otros podían concentrarse en definir cómo estructurar los datos de los productos, logrando un software modular, fácil de entender y listo para seguir creciendo.

